



Binary Diffusion Probabilistic Models

Michel Jean Joseph Donnet

Faculty of Science, University of Geneva

2026-05-26



Outline

Introduction on Diffusion Models

Methodology

Binary Diffusion Models

Binary Diffusion Models for Tabular Data Generation

Results

Bibliography



```
%| refresh: true
%| echo: false
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from utils import *
```



Outline

Introduction on Diffusion Models

Methodology

Binary Diffusion Models

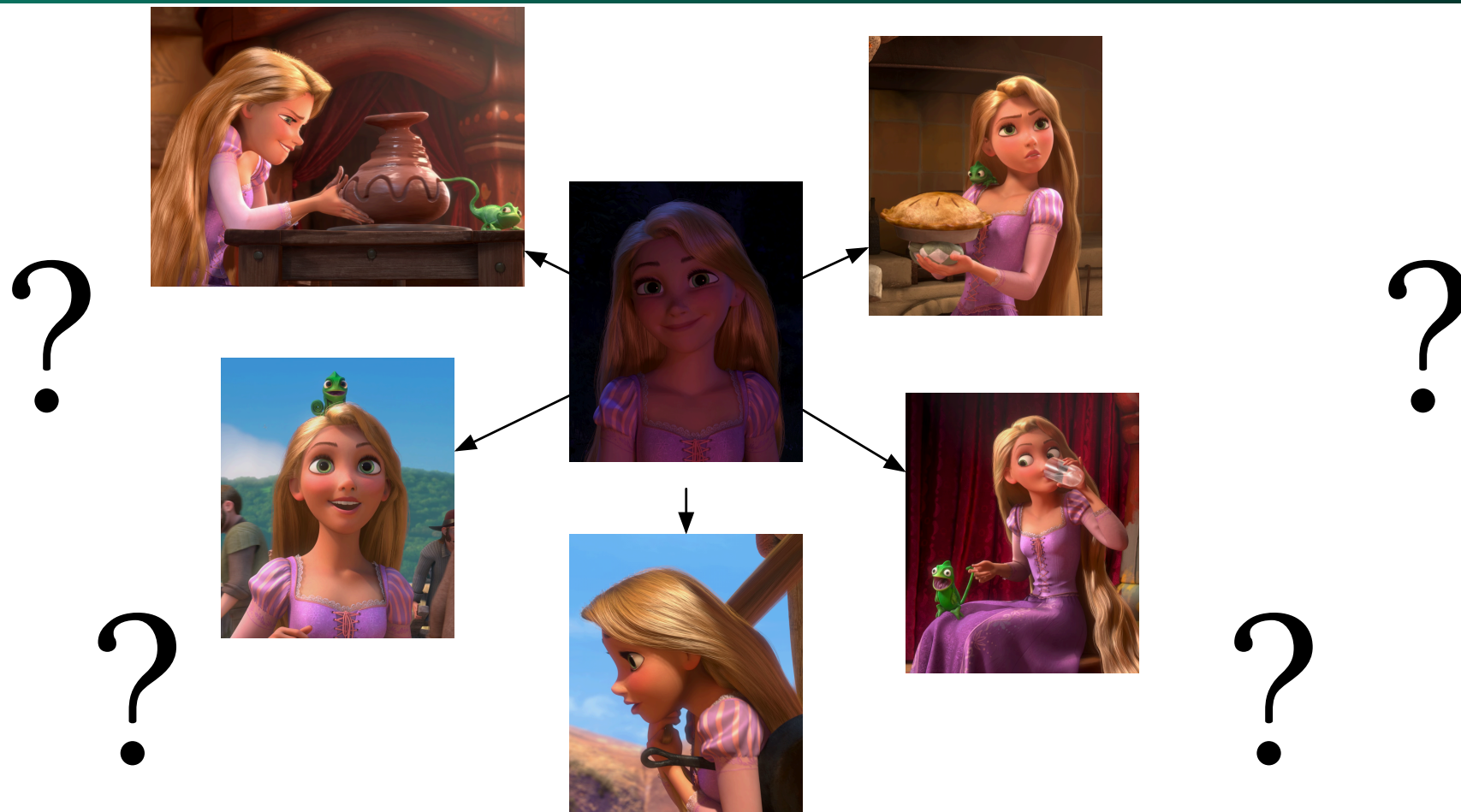
Binary Diffusion Models for Tabular Data Generation

Results

Bibliography



Introduction on Diffusion Models





Introduction on Diffusion Models



?



Outline

Introduction on Diffusion Models

Methodology

Binary Diffusion Models

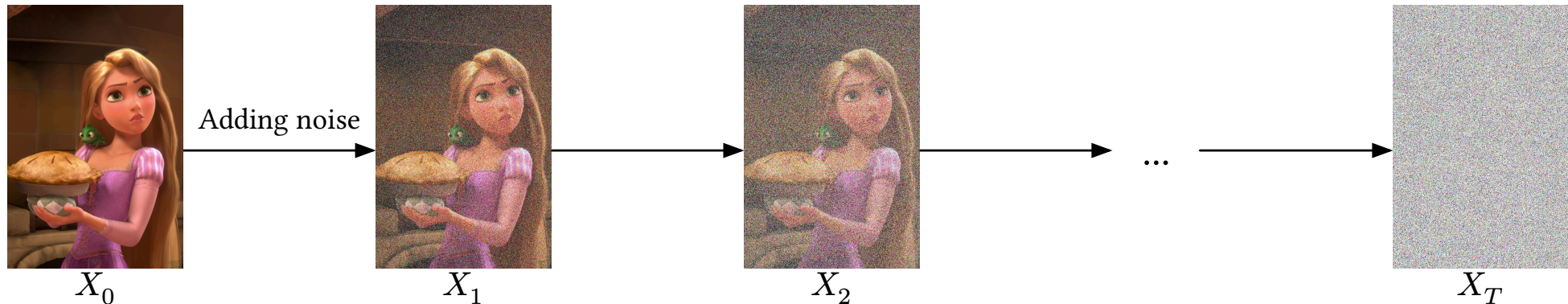
Binary Diffusion Models for Tabular Data Generation

Results

Bibliography

Markov Chain Based Generative Models

Forward Diffusion Process

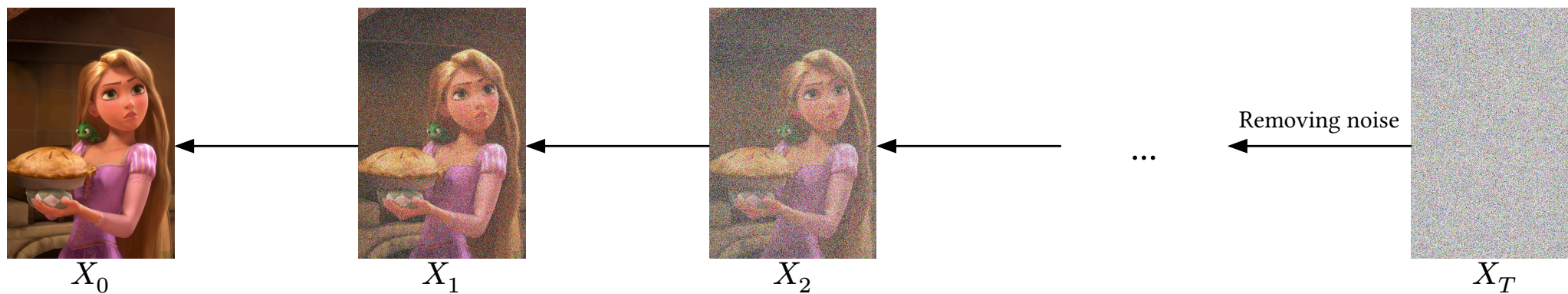


$$X_t = \sqrt{1 - \beta_t} X_{t-1} + \sqrt{\beta_t} \varepsilon_t = \boxed{\sqrt{\bar{\alpha}_t} X_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon}^{[1]}$$

$$q(X_t | X_{t-1}) = \mathcal{N}(X_t; \sqrt{1 - \beta_t} X_{t-1}, \beta_t I)$$

Markov Chain Based Generative Models

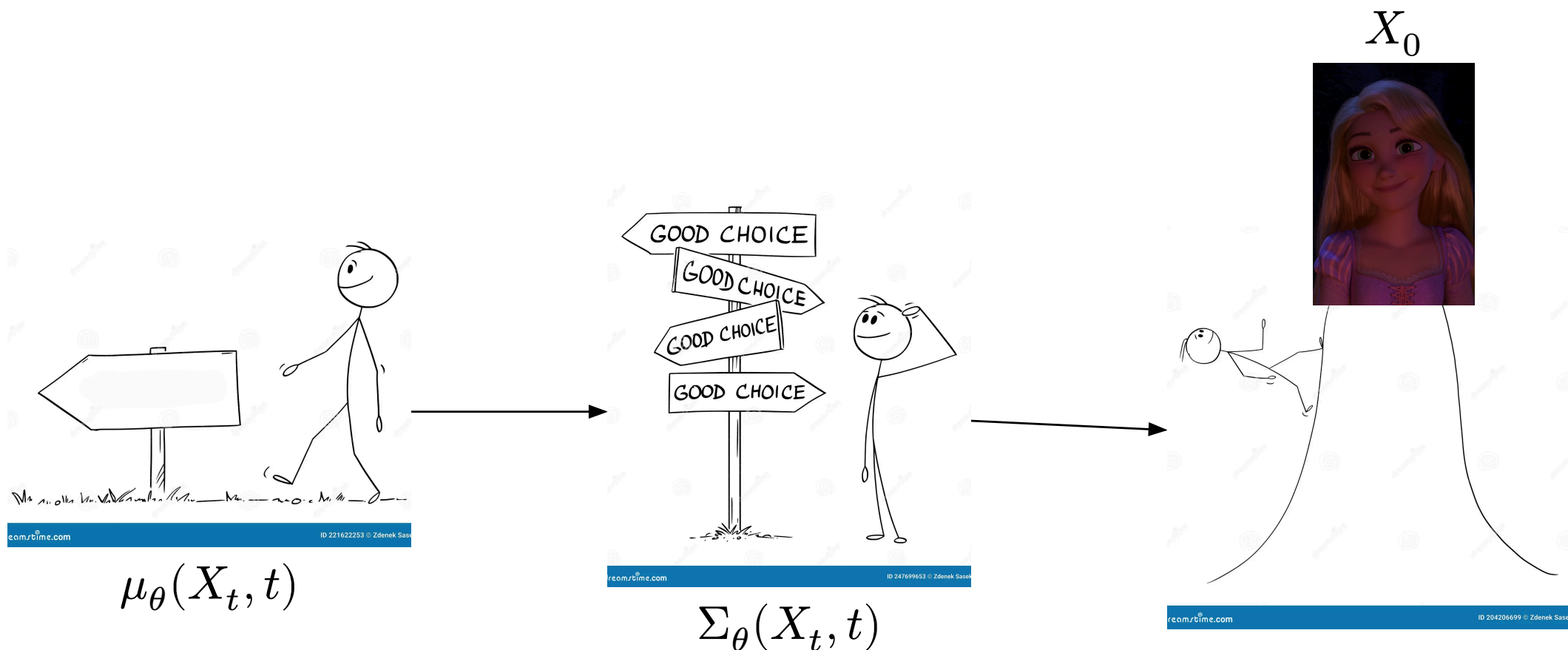
Reverse Diffusion Process



$$p_{\theta}(X_{t-1} \mid X_t) = \mathcal{N}(X_{t-1}; \mu_{\theta}(X_t, t), \Sigma_{\theta}(X_t, t)) \approx q(X_{t-1} \mid X_t)$$

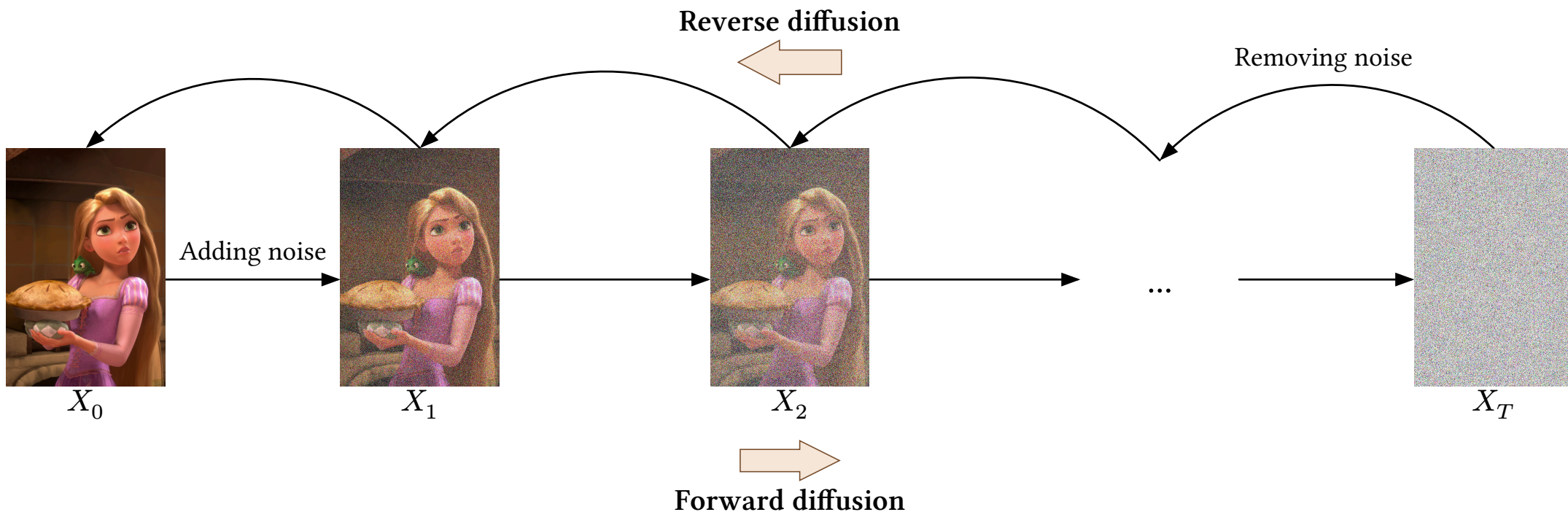
Markov Chain Based Generative Models

Intuitive interpretation



Markov Chain Based Generative Models

Complete process



Outline

Introduction on Diffusion Models

Methodology

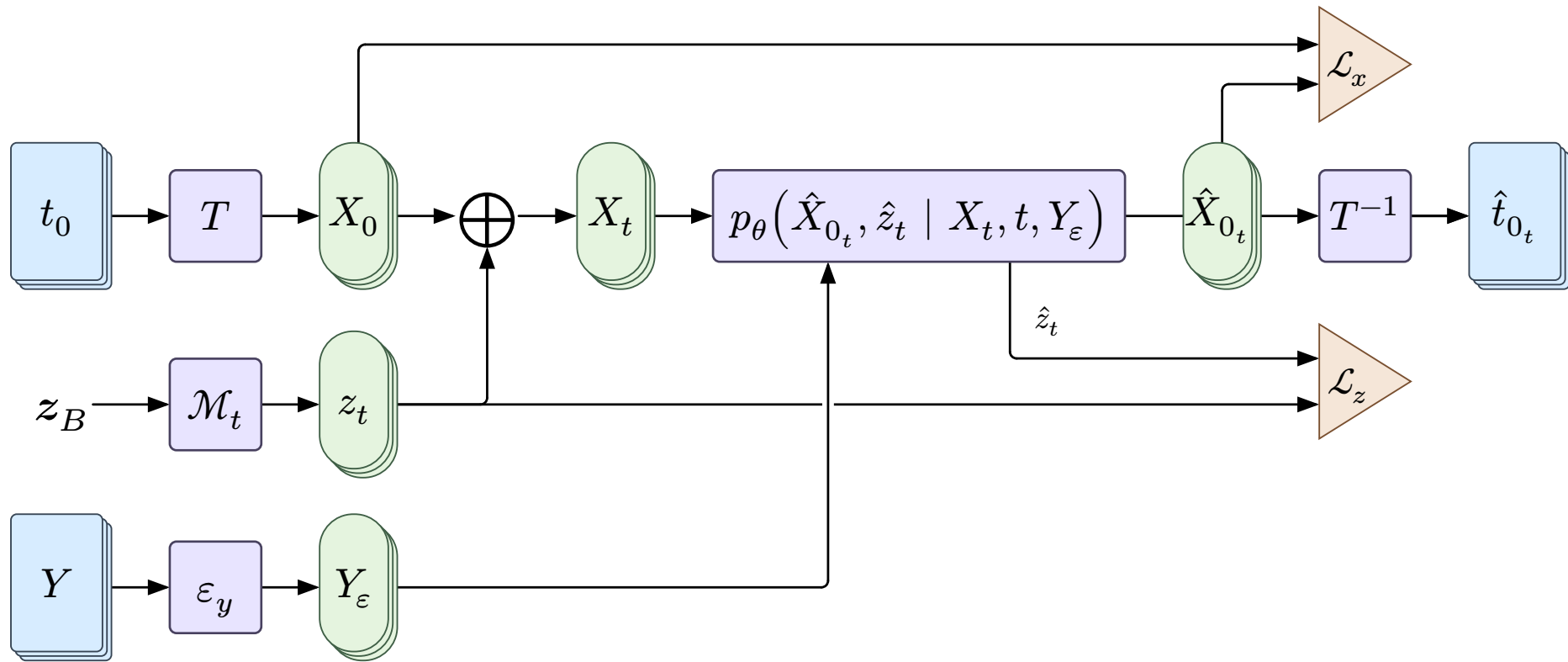
Binary Diffusion Models

Binary Diffusion Models for Tabular Data Generation

Results

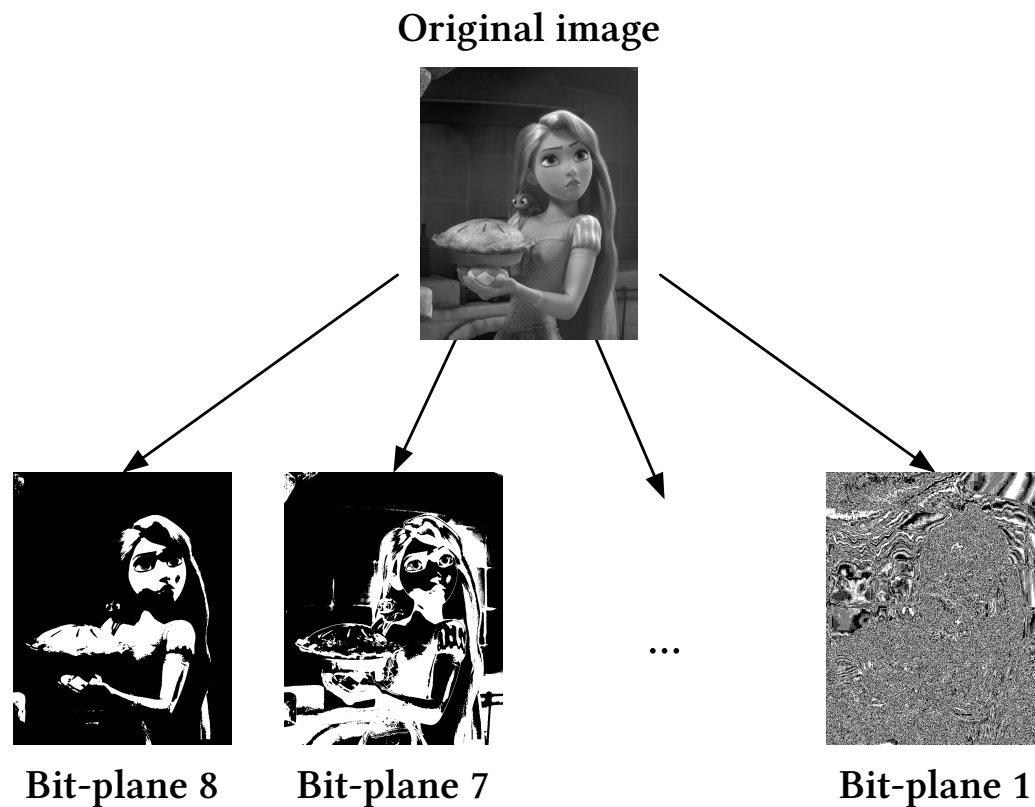
Bibliography

Training the model

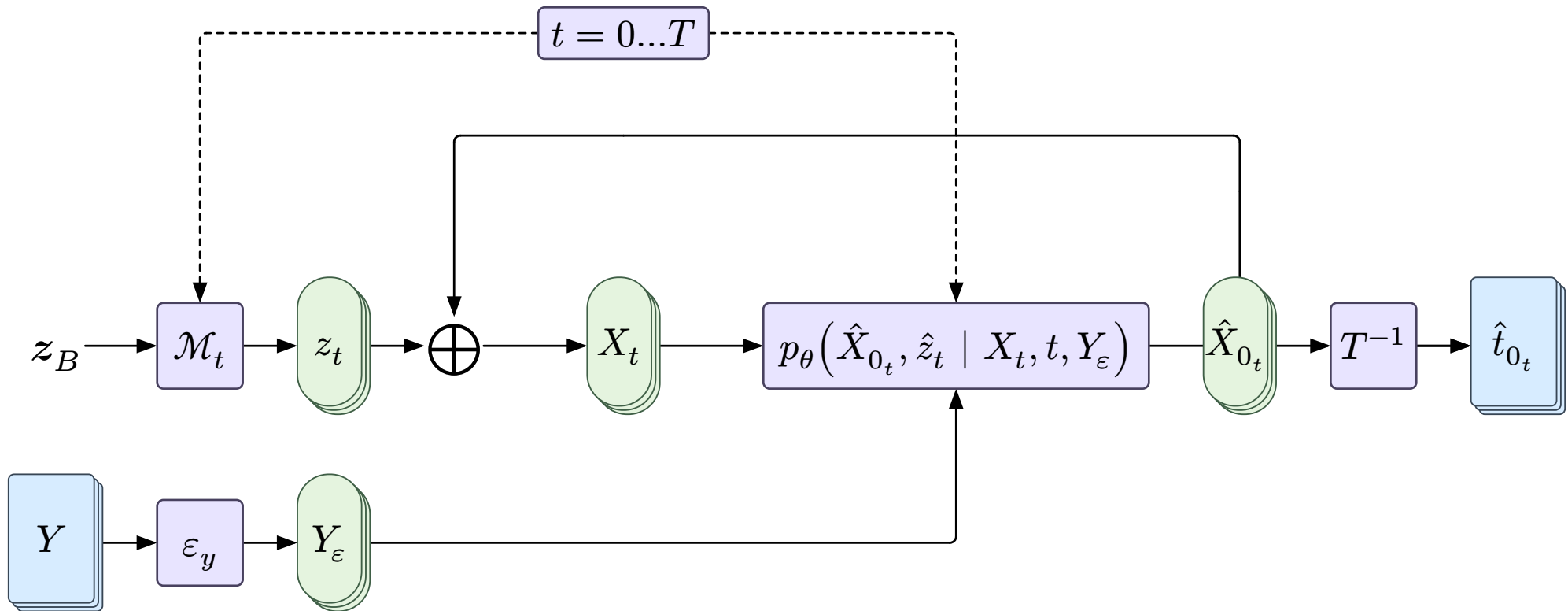


Training the model

First transformation



Sampling from the model





Outline

[Introduction on Diffusion Models](#)

[Methodology](#)

[Binary Diffusion Models](#)

[Binary Diffusion Models for Tabular Data Generation](#)

[Results](#)

[Bibliography](#)



Binary Diffusion Models for Tabular Data Generation

Before working with images which are quite complex data and require more computational resources, we can start by working with tabular data which is simpler and more manageable, and then we can extend the approach to images.

In the paper “Tabular Data Generation using Binary Diffusion” by V. Kinakh and S. Voloshynovskiy [2], the authors propose a binary diffusion model for generating synthetic tabular data, which is a common type of quantized data that can be represented in a binary format.

The model is trained to learn the underlying distribution of the tabular data and can be used to generate new synthetic samples that resemble the original data, which can be useful for various applications such as data augmentation, privacy preservation, and imputation of missing values.



Link with image generation

The process of generating tabular data is similar to the process of generating images, the main difference being the first transformation T and the fact that the data are 1-dimensional instead of 2-dimensional as in the case of images.



First transformation for tabular data

The first transformation T for tabular data can be defined as a simple binary encoding of the original data, where the categorical features are one-hot encoded and the numerical features are binarized using a fixed number of bits to represent the values.

Then, all binary features are concatenated to form a single binary vector that represents the original tabular data, that can be reversed using the inverse transformation T^{-1} to recover the original tabular data from the binary representation.



Architecture of the model

As described in the paper [2], the sampling algorithm for generating synthetic data from the binary diffusion model can be represented as in the .



Architecture of the model

Algorithm 1: Sampling Algorithm

```

1:  $X_t \leftarrow$  random binary tensor ▷ Initialize noisy sample
2:  $Y \leftarrow$  condition/label
3:  $Y_\varepsilon \leftarrow$  apply condition
4:  $p_\theta \leftarrow$  pre-trained denoiser network
5: for  $t \in \{T, \dots, 0\}$  do
6:    $\hat{X}_0, \hat{z}_t \leftarrow p_\theta(\hat{X}_{0_t}, \hat{z}_t \mid X_t, t, Y_\varepsilon)$  ▷ Predict clean sample and latent noise
7:    $\hat{X}_0 \leftarrow \sigma(\hat{X}_0) > \text{threshold}$  ▷ Apply sigmoid and binarize
8:    $\hat{z}_t \leftarrow \sigma(\hat{z}_t) > \text{threshold}$ 
9:    $z_t \leftarrow \text{get\_binary\_noise}(t-1)$  ▷ Generate random binary noise for time step t-1
10:   $X_t \leftarrow \hat{X}_0 \oplus z_t$  ▷ Update sample using XOR
11: end
12: return  $X_t$ 

```



Architecture of the model

Fixed thresholding step

In the paper [2], the authors mention that the model doesn't perform well when increasing the number of time steps T in the reverse diffusion process.

This behavior is quite surprising since increasing the number of time steps T should allow the model to generate higher quality samples by gradually removing the noise from the data, which is a common behavior observed in diffusion models for continuous data.

However, note that in the , the binarization step uses a fixed threshold value that is applied to the predicted clean sample $\hat{X}_0$ to determine the binary values of the generated sample. This fixed thresholding step can be the root cause of the performance degradation.

For instance, if the model is in mode “mask”, the predicted noise $\hat{z}_t$ can have 50% of ones meaning that the model will bring 50% of changes to the data at each step of the reverse diffusion process. This can degrade the quality of the generated samples since the model



Architecture of the model

will be forced to make a large number of changes to the data at each step instead of gradually removing the noise.

Some tests have been done to try to understand the impact of the fixed thresholding step on the performance of the model

Evaluation of the model

To evaluate the performance of the binary diffusion model for tabular data generation, the procedure described in the paper [3] is followed.

The evaluation procedure consists of the following steps:

1. Train the binary diffusion model on the original dataset.
2. Generate a synthetic dataset by sampling from the trained model.
3. Train a machine learning model on the synthetic dataset.
4. Evaluate the performances.



Architecture of the model

In the paper [2], the authors use a Random Forest classifier, a Linear/Logistic Regression classifier, and a Decision Tree classifier to evaluate the performance of the generated synthetic data on a classification and regression tasks, and they report the results in terms of prediction accuracy for classification tasks and mean squared error for regression tasks.

Outline

Introduction on Diffusion Models

Methodology

Binary Diffusion Models

Binary Diffusion Models for Tabular Data Generation

Results

Bibliography

Results

First, I managed to reproduce the results of the paper [2] on the Adult dataset [4].

Then, I tried to improve the performance of the model by bringing some modifications:

- Removing the fixed thresholding
- Correctly applying the guidance during the sampling process
- Fixing the noise addition step in the sampling algorithm

Removing the fixed thresholding step

The first modification was to remove the fixed thresholding step in the sampling algorithm (mode “mask”).

Instead of applying a fixed threshold to binarize the predicted clean sample $\hat{X}_0$, I applied a dynamic threshold that is computed based on the quantity of noise to be removed at each step of the reverse diffusion process.

This dynamic thresholding approach allows the model to adapt the binarization process to the amount of noise that is being removed at each step, which can lead to a gradual removal of the noise and an improvement in the quality of the generated samples as we increase the number of time steps T in the reverse diffusion process.

The shows the results of the evaluation of the model with the dynamic thresholding approach (linear and quadratic schedules) compared to the fixed thresholding approach (constant schedule) for different values of T .

Removing the fixed thresholding step

We can see that the dynamic thresholding approach doesn't improve at all the performance of the model compared to the fixed thresholding approach. And between the two dynamic thresholding approaches, the linear schedule performs worse than the quadratic schedule, which is expected since the model was trained with a quadratic noise schedule.

```
%| echo: false
%| plt-axes.grid: false
%| img-width: 60%
%| label: fig7
fig1 = plot_heatmaps(
    value_col      = "mean",
    x_col          = "n_timesteps",
    y_col          = "schedule",
```


Removing the fixed thresholding step

```
fixed_filters = {"strategy": "mask", "use_t_next": True,
"renoise_factor": 1.0},
# subplot_cols = ["strategy"],
# ncols = 2,
cmap = "RdYlGn",
suptitle = "Schedule manual vs. auto (`const`) for
`strategy=mask` and `use_t_next=True`",
cell_size = (5, 4),
)
plt.show()
```

Indeed, the model already predicts the right amount of noise to be removed at each step, as shown by the following results that compare the quantity of noise in the predicted

Removing the fixed thresholding step

mask with the quantity of noise that should be removed at each step of the reverse diffusion process.

```
Step 800: Ones in mask: 33.6364%, Ones required: 33.6031%  
Step 600: Ones in mask: 20.4545%, Ones required: 20.3957%  
Step 400: Ones in mask: 10.4545%, Ones required: 10.4685%  
Step 200: Ones in mask: 4.0909%, Ones required: 3.8215%  
Step 0: Ones in mask: 0.4332%, Ones required: 0.4545%
```

So this analysis shows that the model predicts the right amount of noise to be removed at each step, which indicates that the fixed thresholding step is not the root cause of the performance degradation when increasing the number of time steps T in the reverse diffusion process.

Correctly applying the guidance during the sampling process

The second modification was to follow the approach described in the paper “Classifier-Free Diffusion Guidance” [5] to guide correctly the sampling process. In fact, the guidance was not correctly applied in the current implementation since it was not following the approach proposed in the paper, even though the goal was to implement the classifier-free guidance as described in the paper [5].

In the paper, the approach proposed is the following:

$$(1 + w) \cdot \varepsilon_c - w \cdot \varepsilon_u$$

Where ε_c is conditioned prediction and ε_u is unconditioned prediction, and w is a scalar value that controls the strength of the guidance.

The conditioned prediction can be seen as the conformity to the desired properties, while the unconditioned prediction can be seen as the diversity of the generated samples, and the guidance allows to find a good balance between these two aspects to generate high-

Correctly applying the guidance during the sampling process

quality samples that match the desired properties while still being diverse and representative of the general distribution of the data.

The current implementation was doing the following:

$$\varepsilon_u + g \cdot (\varepsilon_c - \varepsilon_u)$$

which is equivalent to the approach proposed in the paper [5] but with a different parameterization of the guidance strength, where $g = 1 + w$:

$$\varepsilon_u + (1 + w) \cdot (\varepsilon_c - \varepsilon_u) = (1 + w) \cdot \varepsilon_c - w \cdot \varepsilon_u$$

This modification allows to correctly apply the guidance during the sampling process, but this is only a small fix of -1 in the implementation that doesn't have a significant impact on the performance of the model.

Fixing the noise addition step in the sampling algorithm

The third modification was a small fix in the implementation of the sampling algorithm.

As described in the at line 11, the sample process adds the noise of the step $t - 1$ to the predicted clean sample $\hat{X}_0$ to get the new sample X_t for the next step of the reverse diffusion process.

But in the existent implementation, the noise of the current step t was added instead of the noise of the next step $t - 1$, which can lead to a degradation of the performance of the model.

Indeed, the noise of the current step t is not the correct noise to be added since it corresponds to the noise that was removed at the current step, while the noise of the next step $t - 1$ corresponds to the noise that will be removed at the next step, which is the correct noise to be added to ensure that the model effectively reverses the noise addition process and generates high-quality samples.

Fixing the noise addition step in the sampling algorithm

By adding the correct noise at each step, the amount of noise is now correctly reduced at each step of the reverse diffusion process.

However, as shown in , this modification allows to generate better samples for small values of T but it doesn't allow to improve the performance of the model for larger values of T .

```
%| echo: false
%| label: fig8
%| plt-axes.grid: false
%| grid-columns: (1fr, 0.92fr)
# plot_sensitivity(PARAM_KEYS, DATASETS, PLOT_CONFIG)
# plot_timestep_lines(DATASETS, PLOT_CONFIG)
```

Fixing the noise addition step in the sampling algorithm

```
fig1 = plot_heatmaps(  
    value_col      = "mean",  
    x_col          = "n_timesteps",  
    y_col          = "use_t_next",  
    fixed_filters  = {"renoise_factor": 1.0, "schedule": "const"},  
    subplot_cols   = ["strategy"],  
    ncols          = 2,  
    cmap           = "RdYlGn",  
    suptitle       = "Fixed vs. non-fixed noise addition (`use_t_next`)  
for different strategies",  
    cell_size      = (5, 4),  
)  
plt.show()
```

Fixing the noise addition step in the sampling algorithm

Increasing the noise in the sampling process

The last modification was to increase the noise in the sampling process by multiplying the predicted noise by a renoise factor that is greater than 1, which allows to add more noise at each step of the reverse diffusion process.

This modification allows to better explore the space of possible samples, which results in an improvement of the performance of the model for larger values of T in the reverse diffusion process, as shown in.

```
%| echo: false
%| label: fig9
%| plt-axes.grid: false
%| grid-columns: (1fr, 0.92fr)
# plot_sensitivity(PARAM_KEYS, DATASETS, PLOT_CONFIG)
```


Fixing the noise addition step in the sampling algorithm

```
# plot_timestep_lines(DATASETS, PLOT_CONFIG)

fig1 = plot_heatmaps(
    value_col      = "mean",
    x_col          = "n_timesteps",
    y_col          = "renoise_factor",
    fixed_filters  = {"use_t_next": True, "schedule": "const"},
    subplot_cols   = ["strategy"],
    ncols          = 2,
    cmap           = "RdYlGn",
    suptitle       = "Impact of increasing the noise in the sampling
process (`renoise_factor`) for `use_t_next=True`",
    cell_size      = (5, 4),
```

Fixing the noise addition step in the sampling algorithm

```
)  
plt.show()
```

The shows the best configurations for each strategy compared to the original implementation of the model (baseline) for different values of T in the reverse diffusion process. We can see that the modifications allow to improve the performance of the model of up to 1% for different configurations, which shows that the modifications have a positive impact on the performance of the model, but there is still room for improvement to further enhance the performance of the model for larger values of T in the reverse diffusion process.

```
%| echo: false  
%| label: fig10
```

Fixing the noise addition step in the sampling algorithm

```
%| fig-kind: table  
%| caption: Best configurations vs. original implementation (`baseline`)  
plot_study_table()
```

Here we were dealing with tabular data which is simpler than images, but the same modifications can be applied to the case of image generation to try to improve the performance of the model for larger values of T in the reverse diffusion process.

These modifications can be much more impactful in the case of images since the data are more complex and the noise addition and removal process is more challenging compared to the case of tabular data.

Bibliography

Bibliography

- [1] L. Weng, “What are diffusion models?,” *lilianweng.github.io*, July 2021, [Online]. Available: <https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>
- [2] V. Kinakh and S. Voloshynovskiy, “Tabular Data Generation using Binary Diffusion.” [Online]. Available: <https://arxiv.org/abs/2409.13882>
- [3] V. Borisov, K. Seßler, T. Leemann, M. Pawelczyk, and G. Kasneci, “Language Models are Realistic Tabular Data Generators.” [Online]. Available: <https://arxiv.org/abs/2210.06280>
- [4] B. Becker and R. Kohavi, “Adult.” 1996.
- [5] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance.” [Online]. Available: <https://arxiv.org/abs/2207.12598>